

Answer the Questions

Question 1: What is the purpose of the finally block?

- The purpose of the **finally block** is to execute a piece of code **regardless of whether an exception occurs or not**.

Question 2: How does `int.TryParse()` improve program robustness compared to `int.Parse()`?

- `int.TryParse()` improves program robustness by safely handling invalid input without throwing an exception. It returns `true` if the conversion succeeds and `false` if it fails, allowing the program to handle invalid input gracefully.

Question 3: What exception occurs when trying to access Value on a null `Nullable<T>`?

- Accessing the Value property of a null `Nullable<T>` throws an `InvalidOperationException`.

Question 4: Why is it necessary to check array bounds before accessing elements?

- It is necessary to check array bounds to prevent accessing an invalid index, which would cause an `IndexOutOfRangeException` and potentially terminate the program if the exception is not handled.

Question 5: How is the `GetLength(dimension)` method used in multi-dimensional arrays?

- `GetLength(dimension)` is used to return the number of elements in a specific dimension of a multi-dimensional array. The dimension is zero-based, so 0 refers to the first dimension, 1 to the second, and so on.

Question 6: How does the memory allocation differ between jagged arrays and rectangular arrays?

- Rectangular arrays allocate a fixed-size multi-dimensional structure, while jagged arrays allocate each row separately. This allows jagged arrays to have different row sizes and potentially use memory more efficiently.

Question 7: What is the purpose of nullable reference types in C#?

- Nullable reference types improve code safety by making possible null values explicit and allowing the compiler to warn about potential null-reference errors.

Question 8: What is the performance impact of boxing and unboxing in C#?

- Boxing and unboxing can negatively affect performance because boxing involves heap allocation and copying, while unboxing requires type conversion. Frequent use can increase memory usage and garbage collection overhead.

Question 9: Why must out parameters be initialized inside the method?

- `out` parameters must be initialized inside the method because they are required to have a value before the method returns. Unlike `ref`, they do not need to be initialized before the method is called.

Question 10: Why must optional parameters always appear at the end of a method's parameter list?

- Optional parameters must be at the end so that all required parameters can be supplied without ambiguity when the method is called.

Question 11: How does the null propagation operator prevent `NullReferenceException`?

- The `?.` operator prevents `NullReferenceException` by checking if the object is `null` before accessing its members. If it is `null`, the expression returns `null` instead of throwing an exception.

Question 12: When is a switch expression preferred over a traditional if statement?

- A `switch` expression is preferred when a single value needs to be compared against multiple cases and each case produces a result. It provides a concise and readable alternative to multiple `if-else` statements.

Question 13: What are the limitations of the `params` keyword in method definitions?

- The `params` parameter must be the last parameter, only one `params` parameter is allowed per method, and it must be a single-dimensional array.